

MODERN GENAI ENGINEERING

WITH LANGCHAIN AND STREAMLIT

A ground-up guide to LCEL, tools, agents, memory, retrieval, middleware, and MCP — built with real project separation of concerns, verified against current LangChain 1.x documentation

Beginner → Intermediate → Advanced

Every topic includes a worked exercise with solution and a mini project

Prepared by Fonyuy Gita | SEED — Skills Enhancement and Empowerment Through Digitalization

Table of Contents

How to Use This Guide

This guide assumes basic familiarity with Streamlit (widgets, `session_state`, the chat interface) covered in the companion guide, Modern AI Engineering with Streamlit. Here the focus shifts entirely to LangChain: how it structures model calls, tools, agents, memory, and retrieval, and how to wire all of it into a Streamlit UI without collapsing the project into one unmaintainable file.

This guide is organized into three parts: Beginner, Intermediate, and Advanced. Work through them in order — Part 3's agentic RAG, middleware, and multi-agent chapters all assume the project structure, tool patterns, and memory model established in Parts 1 and 2.

Every chapter follows the same structure:

- Concept explanation with runnable code examples, verified against current LangChain 1.x documentation rather than older tutorials, since this ecosystem changes quickly
- An exercise with a full worked solution and an explanation of why the solution works, not just what it does
- A mini project that extends the chapter's idea into a piece of one running project: `genai_app`, a SEED-flavored GenAI assistant that grows chapter by chapter

A single project structure (introduced in Chapter 2) is used for every mini project after it. This is deliberate: real separation of concerns cannot be taught through one-off snippets. Every later exercise assumes the core/ package layout from Chapter 2 already exists.

A note on accuracy: LangChain has moved fast. Older, still-common patterns you may find elsewhere — `AgentExecutor`, `initialize_agent`, `ConversationBufferMemory` — are legacy as of LangChain 1.0 and are called out explicitly where relevant, rather than taught as the primary pattern. This guide teaches `create_agent`, `LangGraph` checkpoints, and `LCEL` as the current, standard approach.

Prerequisites: comfort with basic Python (functions, classes, type hints, `async/await` at a basic level) and the Streamlit fundamentals from the companion guide. An Anthropic API key is needed from Chapter 1 onward. A local, free HuggingFace embedding model is used for the RAG chapters so no second API key is required to follow along.

Part 1: Beginner — LangChain Foundations and Project Structure

LangChain is not a model, and it is not magic — it is a set of standard interfaces for wiring models, prompts, tools, and data sources together, so you are not rewriting the same plumbing for every provider. This part builds the mental model from first principles: what a Runnable is, how the pipe operator works, and how to structure a real project from day one so it does not collapse into one giant Streamlit file once agents and RAG are added in Part 3.

1. What LangChain Actually Is

The problem LangChain solves

Every LLM provider (Anthropic, OpenAI, Google, local models via Ollama) has a slightly different SDK, a different message format, and a different way of describing tools. Code written directly against one provider's SDK is locked to that provider. LangChain defines a common set of interfaces — chat models, prompts, output parsers, tools, retrievers — so the same application code can run against Claude today and a different model tomorrow with a one-line change.

As of LangChain 1.0 (the current major version), the framework is explicitly positioned as an agent engineering platform: a small, stable core built around one function for building agents (`create_agent`), a standard tool-calling interface, and middleware for customizing agent behavior. Older patterns from LangChain 0.x — `AgentExecutor`, `initialize_agent`, and the legacy memory classes — still work but now live in a separate `langchain-classic` package and are not where new projects should start.

The package layout you actually install

LangChain is split into several packages, and understanding this split explains a lot of confusing import errors beginners hit.

- `langchain-core` — the base abstractions (`Runnable`, `BaseChatModel`, `BaseTool`, messages). Almost everything else depends on this.
- `langchain` — the current agent-building surface: `create_agent`, middleware, prompt utilities. This is the main package for new projects.
- `langchain-anthropic` / `langchain-openai` / `langchain-huggingface` — provider-specific integration packages, installed only for the providers you actually use.
- `langgraph` — the lower-level orchestration engine that `create_agent` is built on top of. You will use it directly for persistence (checkpoints) even in simple projects.
- `langchain-community` — a large grab-bag of third-party integrations (document loaders, some vector stores) maintained by the community rather than the core team.
- `langchain-text-splitters` — chunking utilities used in RAG pipelines, covered in Part 2.

```
pip install langchain langchain-core langchain-anthropic langgraph python-dotenv streamlit
```

The core idea: a Runnable

Nearly everything in LangChain — a chat model, a prompt template, an output parser, a retriever, an entire agent — implements the same `Runnable` interface: `.invoke()` for a single call, `.stream()` for token-by-token output, `.batch()` for many inputs at once, and their async equivalents (`.ainvoke()`, `.astream()`, `.abatch()`). Once you understand `Runnable`, you already know how to call almost anything in LangChain, because the interface does not change based on what the object represents.

```
from langchain.chat_models import init_chat_model

model = init_chat_model("anthropic:claude-sonnet-5")

response = model.invoke("Explain what a Runnable is in one sentence.")
print(response.content)
```

init_chat_model — provider-agnostic initialization

`init_chat_model` is the recommended entry point for creating a chat model in modern LangChain. It takes a string of the form "provider:model-name" and returns an object that behaves identically regardless of provider, which is what lets an application swap models by changing one config value instead of rewriting integration code.

```
from langchain.chat_models import init_chat_model

claude = init_chat_model("anthropic:claude-sonnet-5", temperature=0.7)
# Swapping providers later only changes this one line:
# gpt = init_chat_model("openai:gpt-5", temperature=0.7)
```

Exercise

EXERCISE — Try it yourself first

Install `langchain`, `langchain-core`, and `langchain-anthropic` in a fresh virtual environment. Set your `ANTHROPIC_API_KEY` as an environment variable (do not hardcode it). Use `init_chat_model` to create a model and call `.invoke()` with a question about your own bootcamp's mission (SEED — Skills Enhancement and Empowerment Through Digitalization). Print only the `.content` attribute of the response, not the raw object.

Worked Solution

SOLUTION

```
import os
from dotenv import load_dotenv
from langchain.chat_models import init_chat_model

load_dotenv() # reads ANTHROPIC_API_KEY from a local .env file

model = init_chat_model("anthropic:claude-sonnet-5", temperature=0.3)
response = model.invoke(
    "In two sentences, explain why digital skills training matters for young people in Cameroon."
)
```

```
print(response.content)
```

model.invoke() returns an AIMessage object, not a plain string — it also carries metadata like token usage and the model name. Printing response.content isolates just the text, which is what you almost always want to display to a user.

Mini Project: Provider-Agnostic Model Ping Utility

MINI PROJECT

Build a tiny command-line (not Streamlit yet) utility that proves the provider-agnostic design works, and that you can reason about Runnable objects before adding a UI on top.

Requirements:

- A function `get_model(provider_and_name: str)` that wraps `init_chat_model` and returns the model
- A function `ping(model, question: str) -> str` that calls `.invoke()` and returns only the text content
- Call `ping()` with the same question against two different Claude model strings (e.g. `claude-sonnet-5` and `claude-haiku-4-5`) and print both answers with clear labels
- Wrap each call in a `try/except` that prints a clear error if `ANTHROPIC_API_KEY` is missing, rather than letting a raw exception surface
- Keep this as a single file for now (`ping_test.py`) — Chapter 2 introduces the folder structure this will eventually move into

2. Real Project Structure: Separation of Concerns from Day One

Why this chapter comes before any real feature

The single most common mistake in LangChain tutorials is putting every prompt, every model call, and every Streamlit widget into one `app.py` file. That works for a 20-line demo and becomes unmanageable the moment you add a second chain, a tool, or a retriever. This guide uses one consistent project layout from here forward, and every later mini project is expected to follow it — this is what 'real separation of concerns' means in practice, not just a slogan.

The layout used throughout this guide

```
genai_app/
├── app.py                # Streamlit entry point – thin, no business logic
├── config.py            # constants: model names, chunk sizes, paths
├── .env                 # secrets – never committed
├── .streamlit/
│   └── secrets.toml     # secrets for deployed Streamlit Community Cloud apps
├── core/
│   ├── models.py        # get_model() – all model instantiation lives here
│   ├── prompts.py       # PromptTemplate / ChatPromptTemplate definitions
│   ├── chains.py        # LCEL chains (prompt | model | parser)
│   ├── tools.py         # @tool-decorated functions
│   ├── agents.py        # create_agent() calls, middleware config
│   └── retrieval.py      # loaders, splitters, vector store, retriever
├── pages/
│   ├── 1_Chat.py
│   └── 2_Document_QA.py # Streamlit multi-page UI (from Chapter 5 onward)
├── tests/
│   └── test_chains.py   # tests target core/*, never app.py
└── requirements.txt
```

Why each boundary exists

- `core/models.py` isolates provider setup — if you switch from Anthropic to another provider, exactly one file changes
- `core/prompts.py` keeps prompt text out of application logic, so a non-engineer (a curriculum lead reviewing tone) can review prompts without reading Python control flow
- `core/chains.py` and `core/agents.py` contain zero Streamlit imports — this is what makes them testable with plain pytest and reusable outside the UI (a CLI tool, an API, a Slack bot)
- `app.py` and `pages/` should contain almost no LangChain logic directly — they call functions imported from `core/` and render the result
- `tests/` importing only from `core/` is a deliberate check: if a test needs to import streamlit, a boundary has already been broken

`core/models.py` — the one file that knows about providers

```
# core/models.py
from langchain.chat_models import init_chat_model
from config import DEFAULT_MODEL, DEFAULT_TEMPERATURE

def get_model(model_name: str = DEFAULT_MODEL, temperature: float = DEFAULT_TEMPERATURE):
    return init_chat_model(model_name, temperature=temperature)
```

config.py — the only file with hardcoded values

```
# config.py
DEFAULT_MODEL = "anthropic:claude-sonnet-5"
DEFAULT_TEMPERATURE = 0.5
CHUNK_SIZE = 800
CHUNK_OVERLAP = 100
RETRIEVER_TOP_K = 4
```

Exercise

EXERCISE — Try it yourself first

Restructure the Model Ping Utility from Chapter 1 into this layout: config.py with the two model strings as constants, core/models.py with a get_model() function, and a slim ping_test.py at the root that imports from core.models and runs the same two-model comparison as before.

Worked Solution

SOLUTION

```
# config.py
MODEL_A = "anthropic:claude-sonnet-5"
MODEL_B = "anthropic:claude-haiku-4-5"
TEMPERATURE = 0.3

# core/models.py
from langchain.chat_models import init_chat_model
from config import TEMPERATURE

def get_model(model_name: str):
    return init_chat_model(model_name, temperature=TEMPERATURE)

# ping_test.py
from config import MODEL_A, MODEL_B
from core.models import get_model

def ping(model, question: str) -> str:
    try:
        return model.invoke(question).content
```

```

except Exception as e:
    return f"Error: {e}"

question = "In one sentence, what is a Runnable in LangChain?"
for name in (MODEL_A, MODEL_B):
    model = get_model(name)
    print(f"{name}: {ping(model, question)}")

```

Nothing about the logic changed — only where each piece lives. This is the entire point of separation of concerns: `ping_test.py` no longer knows or cares which provider string format `init_chat_model` expects, because that detail is encapsulated inside `core/models.py`.

Mini Project: Scaffold the Full `genai_app` Project

MINI PROJECT

Generate the complete folder skeleton this guide will build into for the rest of Part 2 and Part 3. Every later mini project assumes this skeleton already exists.

Requirements:

- Create every folder and empty file shown in the project layout above, including an empty `__init__.py` in `core/` and `tests/` so they import as packages
- Write a real `config.py` with `DEFAULT_MODEL`, `DEFAULT_TEMPERATURE`, `CHUNK_SIZE`, `CHUNK_OVERLAP`, and `RETRIEVER_TOP_K`
- Write a real `core/models.py` with `get_model()` as shown
- Write a minimal `app.py` that only imports `streamlit`, sets `st.set_page_config()`, and shows a placeholder `st.write('genai_app scaffold ready')`
- Write a `.gitignore` excluding `.env`, `.streamlit/secrets.toml`, `__pycache__`, and `venv/`
- Write `requirements.txt` pinning `streamlit`, `langchain`, `langchain-core`, `langchain-anthropic`, and `python-dotenv`

3. Prompt Templates and the LCEL Pipe

Why not just use an f-string

An f-string works for a single fixed prompt, but breaks down once you need reusable templates, variable validation, multi-turn message structures, or composition with other Runnables. PromptTemplate (for plain text) and ChatPromptTemplate (for role-based messages) are LangChain's answer, and — being Runnables themselves — they compose with models and parsers using the same pipe syntax.

ChatPromptTemplate — the one you will use almost always

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a concise teaching assistant for a coding bootcamp. "
              "Answer in {max_sentences} sentences or fewer."),
    ("human", "{question}"),
])

# A prompt template is itself a Runnable:
formatted = prompt.invoke({"max_sentences": 2, "question": "What is a list comprehension?"})
print(formatted.to_messages())
```

LCEL — composing Runnables with the pipe operator

LangChain Expression Language (LCEL) is simply Python's `|` operator overloaded so that chaining Runnables together builds a new Runnable, exactly the way Unix pipes chain commands. Each stage's output becomes the next stage's input. This single pattern — prompt | model | parser — is the backbone of most non-agentic LangChain code.

```
from langchain_core.prompts import ChatPromptTemplate
from langchain.chat_models import init_chat_model
from langchain_core.output_parsers import StrOutputParser

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a concise teaching assistant. Answer in {max_sentences} sentences."),
    ("human", "{question}"),
])
model = init_chat_model("anthropic:claude-sonnet-5")
parser = StrOutputParser()

chain = prompt | model | parser  # this line builds a new Runnable

result = chain.invoke({"max_sentences": 2, "question": "What is a list comprehension?"})
print(result)  # a plain string, thanks to StrOutputParser
```

Streaming a chain

Because the whole chain is a Runnable, `.stream()` works on it exactly like it works on a bare model call, yielding pieces of the final output as they become available. This is the same mechanism used later to stream chain output directly into a Streamlit chat bubble.

```
for chunk in chain.stream({"max_sentences": 3, "question": "Explain recursion."}):
    print(chunk, end="", flush=True)
```

Exercise

EXERCISE — Try it yourself first

Build a chain that takes a topic and a target audience (e.g. 'beginners', 'experienced developers') and generates a one-paragraph explanation pitched at that audience. Use a `ChatPromptTemplate` with a system message containing both variables, place it in `core/prompts.py`, and the assembled chain in `core/chains.py`, following the project structure from Chapter 2.

Worked Solution

SOLUTION

```
# core/prompts.py
from langchain_core.prompts import ChatPromptTemplate

explain_prompt = ChatPromptTemplate.from_messages([
    ("system", "You explain technical topics to a specific audience. "
               "Audience: {audience}. Write exactly one paragraph, "
               "matched to that audience's likely background."),
    ("human", "Explain: {topic}"),
])

# core/chains.py
from langchain_core.output_parsers import StrOutputParser
from core.models import get_model
from core.prompts import explain_prompt

def build_explainer_chain():
    model = get_model()
    return explain_prompt | model | StrOutputParser()

# usage
explainer = build_explainer_chain()
print(explainer.invoke({"audience": "beginners", "topic": "REST APIs"}))
print(explainer.invoke({"audience": "experienced developers", "topic": "REST APIs"}))
```

`build_explainer_chain()` is a factory function rather than a module-level chain object, which matters once Streamlit re-runs the script on every interaction — creating the chain inside a cached function (covered properly in Chapter 5) avoids rebuilding the model connection on every keystroke.

Mini Project: Multi-Style Bootcamp Announcement Generator

MINI PROJECT

A reusable LCEL chain for generating training announcements in different tones, directly usable for real SEED bootcamp communications.

Requirements:

- `core/prompts.py`: a `ChatPromptTemplate` with variables for `{audience}`, `{tone}` (e.g. 'formal', 'friendly', 'urgent'), and `{announcement_details}`
- `core/chains.py`: `build_announcement_chain()` returning `prompt | model | StrOutputParser()`
- A small script (`scripts/generate_announcement.py`) that calls the chain three times with the same details but different tones, and prints each result under a clear header
- Confirm the chain works with `.stream()` as well as `.invoke()` by adding a fourth call that streams to the console token by token

4. Output Parsers and Structured Output

Why raw text is not always enough

A chatbot can return plain text, but a real application often needs the model's answer as data: a list of extracted action items, a category label, a JSON object matching a specific schema your downstream code depends on. LangChain has two related tools for this: output parsers (which parse text after the model produces it) and structured output via `with_structured_output` (which constrains what the model produces in the first place, using the provider's native structured-output or tool-calling capability).

with_structured_output — the modern, preferred approach

Define the shape you want as a Pydantic model, then call `.with_structured_output()` on the chat model. LangChain translates the Pydantic schema into the format the underlying provider needs (for Claude, this uses tool calling under the hood) and returns a validated Python object instead of raw text.

```
from pydantic import BaseModel, Field
from langchain.chat_models import init_chat_model

class ActionItem(BaseModel):
    task: str = Field(description="What needs to be done")
    owner: str = Field(description="Who is responsible")
    due_date: str = Field(description="Due date in YYYY-MM-DD format, or 'unspecified'")

class MeetingSummary(BaseModel):
    summary: str = Field(description="One paragraph summary of the meeting")
    action_items: list[ActionItem]

model = init_chat_model("anthropic:claude-sonnet-5")
structured_model = model.with_structured_output(MeetingSummary)

result = structured_model.invoke(
    "Notes: Gita will finish the RAG chapter by Friday. "
    "The team agreed to review the curriculum next Monday, owner TBD."
)
print(result.summary)
for item in result.action_items:
    print(f"- {item.task} ({item.owner}, due {item.due_date})")
```

StrOutputParser and other simple parsers

For cases that do not need full schema validation, `langchain_core.output_parsers` provides lighter parsers: `StrOutputParser` extracts plain text (used throughout Chapter 3), and `JsonOutputParser` parses a model's JSON-formatted text response into a Python dict, which is useful when you want JSON but do not need Pydantic-level validation.

Exercise

EXERCISE — Try it yourself first

Define a Pydantic model `CourseFeedback` with fields: `sentiment` (one of 'positive', 'neutral', 'negative'), `key_points` (a list of strings), and `suggested_action` (a string). Use `with_structured_output` to classify a piece of free-text student feedback into this schema, and print each field on its own line.

Worked Solution**SOLUTION**

```
from pydantic import BaseModel, Field
from typing import Literal
from core.models import get_model

class CourseFeedback(BaseModel):
    sentiment: Literal["positive", "neutral", "negative"]
    key_points: list[str] = Field(description="Short bullet points summarizing the feedback")
    suggested_action: str = Field(description="One concrete action the instructor could take")

model = get_model()
structured = model.with_structured_output(CourseFeedback)

feedback_text = (
    "The CNN module moved too fast for me. I liked the real datasets though, "
    "just wish there was a recap session before the quiz."
)
result = structured.invoke(feedback_text)
print("Sentiment:", result.sentiment)
print("Key points:", result.key_points)
print("Suggested action:", result.suggested_action)
```

`Literal['positive', 'neutral', 'negative']` constrains the `sentiment` field to exactly those three strings at the schema level, so the model is guided toward one of the three options rather than free-form text, and Pydantic will reject a response that doesn't match at validation time.

Mini Project: Structured Feedback Triage Tool**MINI PROJECT**

Extend the feedback classifier into a small batch-processing tool usable by SEED instructors reviewing many feedback submissions at once — this becomes a Streamlit page in Chapter 5.

Requirements:

- `core/chains.py`: a `build_feedback_classifier()` function returning the structured-output model from the exercise

- A list of at least 5 hardcoded sample feedback strings representing a realistic spread of sentiment
- Loop over the list, classify each one, and collect results into a list of dicts (sentiment, key_points, suggested_action, original_text)
- Print a simple summary: how many were positive/neutral/negative, using a plain Python count, not another model call
- Sort and print the negative-sentiment items first, since those need the most immediate instructor attention

5. Your First Streamlit + LangChain Chatbot

Bringing LCEL into the chat UI pattern

This chapter connects the LCEL chain pattern from Chapter 3 to the Streamlit chat widgets (`st.chat_message`, `st.chat_input`, `st.session_state`) you would already know if you worked through the Streamlit-only guide. The only new piece is passing conversation history into the chain correctly using `MessagesPlaceholder`, and caching the chain so it is not rebuilt on every re-run.

MessagesPlaceholder — feeding history into a prompt

```
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful, concise tutor."),
    MessagesPlaceholder("history"),
    ("human", "{input}"),
])
```

Caching the chain in Streamlit

The model client should be created once, not on every re-run. Use `@st.cache_resource`, the same decorator introduced for expensive objects in the pure-Streamlit guide — a LangChain chain is exactly that kind of object.

```
# core/chains.py
import streamlit as st
from langchain_core.output_parsers import StrOutputParser
from core.models import get_model
from core.prompts import chat_prompt  # the ChatPromptTemplate with MessagesPlaceholder

@st.cache_resource
def get_chat_chain():
    model = get_model()
    return chat_prompt | model | StrOutputParser()
```

The full page

```
# pages/1_Chat.py
import streamlit as st
from langchain_core.messages import HumanMessage, AIMessage
from core.chains import get_chat_chain

st.set_page_config(page_title="Chat", page_icon="🗨️")

if "history" not in st.session_state:
    st.session_state.history = []  # list of HumanMessage / AIMessage objects
```

```

chain = get_chat_chain()

for msg in st.session_state.history:
    role = "user" if isinstance(msg, HumanMessage) else "assistant"
    with st.chat_message(role):
        st.markdown(msg.content)

user_input = st.chat_input("Ask something")
if user_input:
    st.session_state.history.append(HumanMessage(content=user_input))
    with st.chat_message("user"):
        st.markdown(user_input)

    with st.chat_message("assistant"):
        full_reply = st.write_stream(
            chain.stream({"input": user_input, "history": st.session_state.history[:-1]})
        )
    st.session_state.history.append(AIMessage(content=full_reply))

```

Exercise

EXERCISE — Try it yourself first

Modify the chat page above so the system message includes a persona chosen from a sidebar selectbox (e.g. 'Encouraging Mentor', 'Direct Code Reviewer'). The persona text needs to reach the prompt, so change `chat_prompt` from a fixed template to a function `build_chat_prompt(persona)` that returns a fresh `ChatPromptTemplate` with the persona baked into the system message.

Worked Solution

SOLUTION

```

# core/prompts.py
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

def build_chat_prompt(persona: str) -> ChatPromptTemplate:
    return ChatPromptTemplate.from_messages([
        ("system", f"You are a tutor with this persona: {persona}. Stay concise."),
        MessagesPlaceholder("history"),
        ("human", "{input}"),
    ])

# core/chains.py
from langchain_core.output_parsers import StrOutputParser
from core.models import get_model
from core.prompts import build_chat_prompt

def build_chat_chain(persona: str):

```

```

model = get_model()
prompt = build_chat_prompt(persona)
return prompt | model | StrOutputParser()

# pages/1_Chat.py (relevant excerpt)
import streamlit as st
persona = st.sidebar.selectbox("Persona", ["Encouraging Mentor", "Direct Code Reviewer"])
chain = build_chat_chain(persona)  # rebuilt per persona choice; not cached, by design

```

This chain is deliberately NOT wrapped in `@st.cache_resource`, because its output depends on the `persona` argument and `st.cache_resource` with no eligible cache key would keep returning the first `persona`'s chain. When a `Runnable`'s construction genuinely depends on user input, rebuilding it on each relevant re-run is correct — caching should be reserved for values that do not change per interaction.

Mini Project: SEED Tutor Chat Page (Part 1 Capstone)

MINI PROJECT

The first real page of the `genai_app` project, combining everything from Part 1: project structure, LCEL, structured feedback classification, and a working chat UI.

Requirements:

- `pages/1_Chat.py`: the persona-based chat page from the exercise, using the project's core/modules exclusively — no LangChain imports directly inside the page file except message classes
- `pages/2_Feedback_Triage.py`: a Streamlit page wrapping the Structured Feedback Triage Tool from Chapter 4 — a `text_area` for pasting feedback, a button to classify it, and the result shown with `st.metric()` for sentiment and a bullet list for `key_points`
- `app.py`: a landing page listing both pages and a one-paragraph description of the whole project's purpose
- Confirm `session_state.history` persists correctly when you navigate from Chat to Feedback Triage and back
- Write one pytest test in `tests/test_chains.py` that calls `build_chat_chain()` with a fixed `persona` and asserts the returned object has an `.invoke` method (a structural check, not a live API call, so tests run without an API key)

Part 2: Intermediate — Tools, Agents, Memory, and Retrieval

Part 1 built chains: fixed pipelines where you decide the steps in advance. Part 2 introduces components that let the model decide what to do next — tools the model can call, agents that loop between reasoning and tool execution, memory that persists across turns, and the document-processing pipeline that RAG depends on. This is where LangChain stops being a nicer way to call a model and starts being a framework for building genuinely autonomous behavior.

6. Tools: Giving the Model Capabilities Beyond Text

What a tool is, precisely

A tool is a Python function wrapped with a schema (name, description, and typed arguments) that a chat model can read and decide to call. The model itself never executes the function — it only requests a call with specific arguments; your code is responsible for actually running the function and returning the result. This distinction matters: tool calling is a two-way contract between the model's judgment and your code's execution.

The @tool decorator

The simplest way to create a tool: decorate a normal Python function with @tool. The function's docstring becomes the tool's description (critical — this is what the model reads to decide when to use it), and its type hints become the argument schema.

```
from langchain_core.tools import tool

@tool
def get_module_deadline(module_name: str) -> str:
    """Look up the submission deadline for a named SEED bootcamp module.

    Args:
        module_name: The module name, e.g. 'ANN', 'CNN', or 'Transfer Learning'.
    """
    deadlines = {
        "ANN": "2026-09-01",
        "CNN": "2026-09-15",
        "Transfer Learning": "2026-09-29",
    }
    return deadlines.get(module_name, "No deadline found for that module.")

# A tool can be called directly, mostly useful for testing:
print(get_module_deadline.invoke({"module_name": "CNN"}))
```

Binding tools to a model directly (without an agent yet)

Before reaching for a full agent, it helps to see the raw mechanism: bind_tools() tells a model which tools are available. The model's response then may contain tool_calls — requests, not results. Your code executes the requested function and sends the result back as a ToolMessage for a final answer.

```
from langchain_core.messages import HumanMessage, ToolMessage
from core.models import get_model

model = get_model().bind_tools([get_module_deadline])

messages = [HumanMessage("When is the CNN module due?")]
ai_response = model.invoke(messages)
```

```

messages.append(ai_response)

for call in ai_response.tool_calls:
    result = get_module_deadline.invoke(call["args"])
    messages.append(ToolMessage(content=result, tool_call_id=call["id"]))

final = model.invoke(messages)
print(final.content)

```

Why you rarely write this loop by hand

The pattern above — call the model, check for tool_calls, execute them, append results, call the model again — is exactly what create_agent automates in Chapter 7. Seeing it manually once is worth doing so the agent loop never feels like a black box.

Exercise

EXERCISE — Try it yourself first

Write a second tool, count_students_in_module(module_name: str) -> str, returning a hardcoded count per module. Bind both tools to a model and manually run the tool-call loop shown above for the question 'How many students are in the CNN module and when is it due?' — note that the model may request both tools in a single response.

Worked Solution

SOLUTION

```

from langchain_core.tools import tool
from langchain_core.messages import HumanMessage, ToolMessage
from core.models import get_model

@tool
def get_module_deadline(module_name: str) -> str:
    """Look up the submission deadline for a named bootcamp module."""
    deadlines = {"ANN": "2026-09-01", "CNN": "2026-09-15", "Transfer Learning": "2026-09-29"}
    return deadlines.get(module_name, "Unknown module")

@tool
def count_students_in_module(module_name: str) -> str:
    """Look up how many students are enrolled in a named bootcamp module."""
    counts = {"ANN": 24, "CNN": 21, "Transfer Learning": 19}
    return str(counts.get(module_name, 0))

tools = [get_module_deadline, count_students_in_module]
model = get_model().bind_tools(tools)
tools_by_name = {t.name: t for t in tools}

```

```

messages = [HumanMessage("How many students are in the CNN module and when is it due?")]
ai_response = model.invoke(messages)
messages.append(ai_response)

for call in ai_response.tool_calls:
    tool_fn = tools_by_name[call["name"]]
    result = tool_fn.invoke(call["args"])
    messages.append(ToolMessage(content=result, tool_call_id=call["id"]))

final = model.invoke(messages)
print(final.content)

```

tools_by_name maps each tool's .name attribute to the tool object itself, so the loop can dispatch to the correct function purely from call['name'] returned by the model, without an if/elif chain — the same dispatch pattern scales cleanly as more tools are added.

Mini Project: Bootcamp Operations Toolkit (Tools Only, No Agent Yet)

MINI PROJECT

Build the full tool layer for an operations assistant, kept in core/tools.py, that Chapter 7's agent will consume directly. Separating tool definitions from agent wiring is itself a separation-of-concerns exercise.

Requirements:

- core/tools.py: at least 4 tools — module deadline lookup, student count lookup, a prerequisite checker (which module must be completed before another), and a room/session schedule lookup
- Every tool has a clear, specific docstring, since that text is what the model uses to decide when to call it — vague docstrings are a common real-world bug
- A tools_by_name dict built once in core/tools.py and exported for reuse
- A manual test script (scripts/test_tools_manual.py) that runs the bind_tools loop for 3 different natural-language questions, each requiring a different combination of tools
- Confirm what happens when you ask a question that needs no tool at all (e.g. 'What is a neural network?') — the response should have an empty tool_calls list, and your loop should handle that gracefully without erroring

7. Agents with create_agent

What create_agent automates

create_agent is the standard, current way to build an agent in LangChain 1.0. It wraps the exact model-call / tool-execution / model-call loop from Chapter 6 into a single object, built on LangGraph under the hood, and adds a clean interface for system prompts, middleware, structured output, and persistence. It replaces the older AgentExecutor and initialize_agent patterns from LangChain 0.x, which are now considered legacy.

The minimal agent

```
from langchain.agents import create_agent
from core.tools import get_module_deadline, count_students_in_module

agent = create_agent(
    model="anthropic:claude-sonnet-5",
    tools=[get_module_deadline, count_students_in_module],
    system_prompt="You are an operations assistant for a coding bootcamp. Use tools to answer factual questions accurately.",
)

result = agent.invoke({"messages": [{"role": "user", "content": "When is the CNN module due?"}]})
print(result["messages"][-1].content)
```

The agent's input and output shape

An agent built with create_agent takes a dict with a "messages" key (a list of message dicts or message objects) and returns a dict with the same key, containing the full updated message list — including every intermediate tool call and tool result, not just the final answer. This full trace is what Chapter 13's UI uses to show tool use transparently.

Streaming an agent's steps

agent.stream() yields state updates as the agent progresses through its loop (model call, tool call, model call again), rather than waiting for the whole run to finish. This is the mechanism used to show 'thinking' progress in a Streamlit UI, covered fully in Chapter 14.

```
for step in agent.stream({"messages": [{"role": "user", "content": "How many students are in ANN?"}]}):
    print(step)  # each step is a dict keyed by the node that just ran
```

Exercise

EXERCISE — Try it yourself first

Take the 4-tool Bootcamp Operations Toolkit from Chapter 6 and wire it into a `create_agent` call with a clear `system_prompt`. Ask it a question that requires chaining two tools (for example: 'Can a student who hasn't finished ANN start CNN, and how many students are already in CNN?') and print the final answer plus the total number of messages in the returned trace.

Worked Solution

SOLUTION

```
from langchain.agents import create_agent
from core.tools import (
    get_module_deadline, count_students_in_module,
    check_prerequisite, get_schedule,
)

agent = create_agent(
    model="anthropic:claude-sonnet-5",
    tools=[get_module_deadline, count_students_in_module, check_prerequisite,
get_schedule],
    system_prompt=(
        "You are an operations assistant for the SEED bootcamp. "
        "Use the available tools to answer factual questions. "
        "If a question needs more than one tool, call them in the order that makes
sense."
    ),
)

result = agent.invoke({
    "messages": [{
        "role": "user",
        "content": "Can a student who hasn't finished ANN start CNN, and how many
students are already in CNN?"
    }]
})

print(result["messages"][-1].content)
print(f"Total messages in trace (including tool calls): {len(result['messages'])}")
```

The agent decides on its own, based on the system prompt and each tool's docstring, to call `check_prerequisite` and `count_students_in_module` in whatever order it judges necessary — this is the core difference from Chapter 6's manual loop, where the sequence of calls was fixed by your code rather than chosen by the model.

Mini Project: Bootcamp Operations Agent (Streamlit Page)

MINI PROJECT

Wrap the `create_agent` operations assistant in a real Streamlit chat page, following the project structure — `core/agents.py` builds the agent, the page only renders it.

Requirements:

- `core/agents.py`: a `build_operations_agent()` function (cached with `@st.cache_resource`) that returns the `create_agent` instance with all 4 tools and a clear system prompt
- `pages/3_Operations_Agent.py`: a chat interface (`chat_message` + `chat_input`) that calls `agent.invoke()` and displays only the final assistant message by default
- A sidebar toggle 'Show tool calls' that, when enabled, also prints any intermediate `ToolMessage` content from the returned messages list in an `st.expander` below the final answer
- Handle the case where the agent's response contains no tool calls at all (a purely conversational question) without any special-casing in the page code — this should already work if `core/agents.py` is built correctly
- Wrap `agent.invoke()` in `try/except` with a clear `st.error()` on failure

8. Memory and Persistence Across Turns

Why session_state alone is not enough for agents

Chapter 5's chat page manually appended messages to `st.session_state.history` and passed them into the chain on every call. That works for a simple chain, but `create_agent` expects to manage conversation state itself through a checkpointer, and doing so unlocks capabilities `session_state` alone does not have: persistence across app restarts (with a database-backed checkpointer), multiple independent conversation threads, and time-travel debugging.

A note on deprecated memory classes

Older LangChain tutorials use `ConversationBufferMemory`, `ConversationBufferWindowMemory`, or `ConversationSummaryMemory` from `langchain.memory`. As of LangChain 0.3.1, all of these are deprecated in favor of LangGraph's checkpointing system. They still import and run (via `langchain-classic`) but should not be the starting point for a new project — this guide does not use them.

Checkpointers and thread_id

A checkpointer saves the agent's full state after every step, keyed by a `thread_id` you choose. Passing the same `thread_id` on a later call resumes that exact conversation; a new `thread_id` starts a fresh one. `InMemorySaver` keeps state only for the life of the process (fine for local development); production deployments typically swap in a database-backed checkpointer without changing any other code.

```
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents import create_agent
from core.tools import get_module_deadline

checkpointer = InMemorySaver()

agent = create_agent(
    model="anthropic:claude-sonnet-5",
    tools=[get_module_deadline],
    system_prompt="You are a helpful bootcamp assistant.",
    checkpointer=checkpointer,
)

config = {"configurable": {"thread_id": "student-42-session"}}

agent.invoke({"messages": [{"role": "user", "content": "When is the CNN module due?"}]},
config)
# A later call with the SAME thread_id remembers the earlier turn:
result = agent.invoke({"messages": [{"role": "user", "content": "And what about ANN?"}]},
config)
print(result["messages"][-1].content)
```

Mapping thread_id to a Streamlit session

In a Streamlit app, generate one `thread_id` per browser session (store it in `st.session_state` the first time the page loads) so each visitor gets an independent, persistent conversation without your code manually replaying history into every call.

```
import uuid
import streamlit as st

if "thread_id" not in st.session_state:
    st.session_state.thread_id = str(uuid.uuid4())

config = {"configurable": {"thread_id": st.session_state.thread_id}}
# Pass config into every agent.invoke() call for this session
```

Exercise

EXERCISE — Try it yourself first

Take the operations agent from Chapter 7 and add an `InMemorySaver` checkpointer. Run two `invoke()` calls with the same `thread_id` where the second question depends on the first ('When is CNN due?' then 'How many days from today is that?'), and a third call with a different `thread_id` asking the second question alone, to confirm it fails to answer correctly without the missing context — proving the isolation between threads.

Worked Solution

SOLUTION

```
from datetime import date
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents import create_agent
from core.tools import get_module_deadline

checkpointer = InMemorySaver()
agent = create_agent(
    model="anthropic:claude-sonnet-5",
    tools=[get_module_deadline],
    system_prompt=f"You are a bootcamp assistant. Today's date is {date.today().isoformat()}.",
    checkpointer=checkpointer,
)

thread_a = {"configurable": {"thread_id": "thread-a"}}
agent.invoke({"messages": [{"role": "user", "content": "When is the CNN module due?"}]}, thread_a)
follow_up = agent.invoke({"messages": [{"role": "user", "content": "How many days from today is that?"}]}, thread_a)
print("Thread A follow-up:", follow_up["messages"][-1].content)
```

```
thread_b = {"configurable": {"thread_id": "thread-b"}}
isolated = agent.invoke({"messages": [{"role": "user", "content": "How many days from
today is that?"}]}, thread_b)
print("Thread B (no prior context):", isolated["messages"][-1].content)
```

Thread A's second call succeeds because the checkpointer replays the full prior message list for thread-a before adding the new question, so the model can resolve 'that' to the CNN deadline. Thread B has no saved state under thread-b, so 'that' has nothing to refer to — the model will either ask for clarification or state it lacks context, which is the correct, expected behavior of isolated threads.

Mini Project: Persistent Multi-Session Operations Agent

MINI PROJECT

Upgrade the Operations Agent Streamlit page from Chapter 7 to use real thread-based memory instead of resending full history manually.

Requirements:

- core/agents.py: add checkpointer=InMemorySaver() to build_operations_agent()
- pages/3_Operations_Agent.py: generate a UUID thread_id once per session using st.session_state, and pass the config dict into every agent.invoke() call
- A sidebar button 'Start new conversation' that generates a fresh thread_id (via a callback) and clears the visible chat history, without restarting the Streamlit app
- Display the current thread_id (or just its first 8 characters) in small text in the sidebar, useful for debugging during development
- Confirm, by testing manually, that clicking 'Start new conversation' truly forgets prior context — ask a follow-up question that depended on earlier context and confirm the agent no longer has it

9. Document Loading, Splitting, and Embeddings

The first half of any RAG pipeline

Retrieval-Augmented Generation depends on turning raw documents into small, searchable pieces. This chapter covers the ingestion side only — loading and chunking — and the next chapter covers turning chunks into a searchable index. Getting ingestion right (sensible chunk sizes, sensible overlap) has more impact on RAG answer quality than almost any other single decision in the pipeline.

Document loaders

A loader reads a source (a file, a URL, a database) and produces a list of Document objects, each with `page_content` (the text) and `metadata` (like source filename or page number).

`langchain_community.document_loaders` has loaders for most common formats.

```
from langchain_community.document_loaders import PyPDFLoader, TextLoader

pdf_loader = PyPDFLoader("curriculum/field_guide.pdf")
pdf_docs = pdf_loader.load()  # one Document per page, with page number in metadata

text_loader = TextLoader("curriculum/module_notes.txt")
text_docs = text_loader.load()

print(f"Loaded {len(pdf_docs)} pages from the PDF")
print(pdf_docs[0].metadata)  # e.g. {'source': '...', 'page': 0}
```

Splitting into chunks

A full document is almost always too large to embed and retrieve usefully as one unit.

`RecursiveCharacterTextSplitter` is the standard general-purpose splitter: it tries to split on paragraph breaks first, then sentences, then words, only falling back to a hard character cut when necessary, which keeps chunks more semantically coherent than a naive fixed-size cut.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
from config import CHUNK_SIZE, CHUNK_OVERLAP

splitter = RecursiveCharacterTextSplitter(
    chunk_size=CHUNK_SIZE,
    chunk_overlap=CHUNK_OVERLAP,
    add_start_index=True,  # records each chunk's character offset in metadata
)

chunks = splitter.split_documents(pdf_docs)
print(f"{len(pdf_docs)} pages became {len(chunks)} chunks")
print(chunks[0].page_content[:200])
```

Embeddings — turning text into vectors

An embedding model converts text into a fixed-length vector of numbers such that semantically similar text produces nearby vectors. Anthropic does not offer its own embeddings API; common choices are Voyage AI (Anthropic's recommended embeddings partner) for production quality, or a local HuggingFace sentence-transformers model for a free, offline-capable setup, which this guide uses for the runnable examples so no second API key is required.

```
from langchain_huggingface import HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")

vector = embeddings.embed_query("What is the CNN module deadline?")
print(f"Embedding has {len(vector)} dimensions")
```

Exercise

EXERCISE — Try it yourself first

Load a .txt file of your choice (any curriculum notes work), split it with a `chunk_size` of 500 and `chunk_overlap` of 50, and print: the total number of chunks, the character length of the first and last chunk, and confirm that the last 50 characters of `chunk[0]` appear at the start of `chunk[1]` (proving the overlap is working).

Worked Solution

SOLUTION

```
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

loader = TextLoader("curriculum/module_notes.txt")
docs = loader.load()

splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks = splitter.split_documents(docs)

print(f"Total chunks: {len(chunks)}")
print(f"First chunk length: {len(chunks[0].page_content)}")
print(f>Last chunk length: {len(chunks[-1].page_content)}")

overlap_check = chunks[0].page_content[-50:]
print("Overlap present in chunk 2:", overlap_check in chunks[1].page_content)
```

RecursiveCharacterTextSplitter does not guarantee an exact character-for-character overlap match in every case, because it prefers splitting on natural boundaries (paragraphs, sentences) over hitting chunk_overlap precisely — the check may show partial rather than exact overlap, which is expected behavior, not a bug.

Mini Project: Curriculum Ingestion Pipeline

MINI PROJECT

Build the reusable ingestion module that Chapter 10's vector store will consume, following separation of concerns strictly: this module knows nothing about embeddings or retrieval, only about turning files into chunks.

Requirements:

- `core/retrieval.py`: a function `load_and_split(file_path: str) -> list[Document]` that picks `PyPDFLoader` or `TextLoader` based on the file extension, then splits with the project's `CHUNK_SIZE` and `CHUNK_OVERLAP` from `config.py`
- Raise a clear `ValueError` for unsupported extensions rather than letting an unrelated exception surface
- A small script (`scripts/ingest_test.py`) that runs `load_and_split` on 2 different files (one `.txt`, one `.pdf` if available) and prints chunk counts for each
- Add a docstring to `load_and_split` explaining what it returns and what metadata each chunk carries — this becomes important documentation once Chapter 11 shows retrieved sources to end users

10. Vector Stores and Retrievers

From chunks to a searchable index

A vector store holds embedded chunks and can quickly find the chunks most similar to a new query vector. LangChain provides a consistent interface across many vector store backends (Chroma, FAISS, Pinecone, and others), plus a lightweight InMemoryVectorStore built into langchain-core for development and small datasets with no external database required.

Building an in-memory vector store

```
from langchain_core.vectorstores import InMemoryVectorStore
from langchain_huggingface import HuggingFaceEmbeddings
from core.retrieval import load_and_split

embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
chunks = load_and_split("curriculum/field_guide.pdf")

vector_store = InMemoryVectorStore(embeddings)
vector_store.add_documents(chunks)

results = vector_store.similarity_search("What is transfer learning?", k=3)
for doc in results:
    print(doc.page_content[:150], "...")
    print(doc.metadata)
```

A persistent option: Chroma

InMemoryVectorStore loses its index every time the process restarts, which is fine for a demo but not for a real deployed app that should not re-embed a large document set on every restart. Chroma persists to disk with almost the same API.

```
from langchain_chroma import Chroma

vector_store = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory="./chroma_db",
)
# On a later run, reload without re-embedding:
# vector_store = Chroma(persist_directory="./chroma_db", embedding_function=embeddings)
```

Retrievers — the Runnable wrapper around a vector store

as_retriever() turns a vector store into a Retriever, which is itself a Runnable (so it composes with the rest of LangChain the same way models and prompts do) and standardizes the search interface regardless of which vector store backend you chose.

```
retriever = vector_store.as_retriever(search_kwargs={"k": 4})
docs = retriever.invoke("What is transfer learning?") # same call signature as any Runnable
```

Exercise

EXERCISE — Try it yourself first

Build an InMemoryVectorStore from the chunks produced in Chapter 9's exercise. Run 3 different queries against it with `.similarity_search()`, and for each, print the top result's first 100 characters plus its similarity score using `similarity_search_with_score` instead of `similarity_search`.

Worked Solution

SOLUTION

```
from langchain_core.vectorstores import InMemoryVectorStore
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

loader = TextLoader("curriculum/module_notes.txt")
docs = loader.load()
chunks = RecursiveCharacterTextSplitter(chunk_size=500,
chunk_overlap=50).split_documents(docs)

embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
vector_store = InMemoryVectorStore(embeddings)
vector_store.add_documents(chunks)

queries = ["What is a CNN?", "How is the dataset split?", "What is the grading policy?"]
for q in queries:
    top_results = vector_store.similarity_search_with_score(q, k=1)
    doc, score = top_results[0]
    print(f"Query: {q}")
    print(f"Score: {score:.4f}")
    print(f"Top match: {doc.page_content[:100]}...")
```

similarity_search_with_score returns tuples of (Document, score) instead of bare documents. Depending on the vector store's distance metric, a lower score can mean more similar (distance) or a higher score can mean more similar (cosine similarity) — always check which convention a given backend uses before assuming 'higher is better'.

Mini Project: Persistent Curriculum Vector Store

MINI PROJECT

Complete the RAG ingestion pipeline by adding the vector store layer, wired into the ingestion module from Chapter 9, forming the retrieval foundation the agentic RAG tool in Chapter 11 will consume.

Requirements:

- `core/retrieval.py`: extend with `build_vector_store(file_paths: list[str])` using Chroma with a `persist_directory` from `config.py`, calling `load_and_split` on each path and combining all chunks before embedding
- `core/retrieval.py`: a `get_retriever(k: int = None)` function that loads the persisted Chroma store (falling back to `RETRIEVER_TOP_K` from `config.py` if `k` is not given) and returns `.as_retriever()`
- A one-time setup script (`scripts/build_index.py`) that ingests every file in a `curriculum/` folder and reports total chunk count and time taken
- Confirm re-running the Streamlit app does not re-embed the documents by adding a print statement inside `build_vector_store` and observing it does NOT appear on a normal app re-run, only when `scripts/build_index.py` is run explicitly

Part 3: Advanced — Agentic RAG, Middleware, MCP, and Production Systems

Part 2 built the individual components: tools, agents, memory, and a retrieval pipeline. Part 3 assembles them into the kind of system real GenAI products are built from — an agent that retrieves its own context, guardrails that keep it safe and controllable, a standard protocol (MCP) for plugging in external tool servers without writing custom adapters, and the multi-agent and production patterns that separate a working demo from a system a team can maintain.

11. Agentic RAG: The Retriever as a Tool

Why RAG belongs inside an agent, not a fixed chain

An older RAG pattern always retrieves documents for every single question, whether or not the question needs them, using a fixed chain like `create_retrieval_chain`. The modern, more flexible pattern gives the retriever to an agent as a tool, so the model decides, per question, whether to search the knowledge base at all, and can combine retrieval with other tools (like the operations tools from Chapter 6) in the same conversation.

Wrapping a retriever as a tool

```
from langchain_core.tools import tool
from core.retrieval import get_retriever

retriever = get_retriever()

@tool
def search_curriculum(query: str) -> str:
    """Search the SEED curriculum documents for information relevant to the query.
    Use this whenever a question is about course content, deadlines described in
    the materials, or technical topics covered in the bootcamp.

    Args:
        query: The search query, ideally the user's question or a paraphrase of it.
    """
    docs = retriever.invoke(query)
    if not docs:
        return "No relevant material found in the curriculum documents."
    return "\n\n--\n\n".join(
        f"[Source: {d.metadata.get('source', 'unknown')}] \n{d.page_content}"
        for d in docs
    )
```

Giving it to an agent

```
from langchain.agents import create_agent
from core.tools import get_module_deadline, count_students_in_module

agent = create_agent(
    model="anthropic:claude-sonnet-5",
    tools=[search_curriculum, get_module_deadline, count_students_in_module],
    system_prompt=(
        "You are a bootcamp assistant. Use search_curriculum for questions about "
        "course content. Use the other tools for operational questions. "
        "Cite the source file when you use retrieved material."
    ),
)
```

```
result = agent.invoke({"messages": [{"role": "user", "content": "What is transfer learning,
according to our materials?"}]}))
print(result["messages"][-1].content)
```

Showing sources back to the user

Because `search_curriculum`'s return value is plain text embedded in the tool's `ToolMessage`, the source citations the tool includes (`[Source: ...]`) flow naturally into the model's context and often into its final answer if the system prompt asks for citation. For a UI that shows sources separately from the answer (as the mini project below requires), inspect the `ToolMessage` objects in the returned messages list directly rather than relying on the model to always repeat them.

Exercise

EXERCISE — Try it yourself first

Modify `search_curriculum` so it returns 'No relevant material found' when the retriever's top result has a similarity score below a reasonable threshold, instead of always returning whatever was retrieved even if barely related. Use `similarity_search_with_score` directly instead of the plain retriever for this tool.

Worked Solution

SOLUTION

```
from langchain_core.tools import tool
from core.retrieval import get_vector_store

vector_store = get_vector_store()
SCORE_THRESHOLD = 1.0 # tune per embedding model and distance metric; lower = more
similar for Chroma's default L2 distance

@tool
def search_curriculum(query: str) -> str:
    """Search the SEED curriculum documents for information relevant to the query."""
    results = vector_store.similarity_search_with_score(query, k=4)
    relevant = [(doc, score) for doc, score in results if score <= SCORE_THRESHOLD]
    if not relevant:
        return "No sufficiently relevant material found in the curriculum documents."
    return "\n\n--\n\n".join(
        f"[Source: {doc.metadata.get('source', 'unknown')}],\n"
        f"score={score:.3f}]\n{doc.page_content}"
        for doc, score in relevant
    )
```

Filtering by score prevents a common RAG failure mode: retrieving the 'closest' chunks even when none of them are actually relevant, because a vector search always returns the k nearest neighbors regardless of how

far away they are. The threshold value itself is empirical — it depends on the embedding model and distance metric and should be tuned by testing real queries against your actual document set, not assumed from a tutorial.

Mini Project: Agentic Curriculum Assistant with Cited Sources

MINI PROJECT

Combine the retriever tool, the operations tools, and a full chat UI into one Streamlit page that shows retrieved sources transparently — the centerpiece feature of the whole guide.

Requirements:

- `core/tools.py`: add the score-filtered `search_curriculum` tool from the exercise
- `core/agents.py`: `build_curriculum_agent()` combining `search_curriculum` with the Chapter 6 operations tools and a checkpoint (Chapter 8 pattern)
- `pages/4_Curriculum_Assistant.py`: a chat page where, after each assistant turn, any `ToolMessage` with content starting with '[Source:]' is shown in a collapsed `st.expander` labeled 'Sources consulted', separate from the main answer bubble
- If `search_curriculum` returns 'No sufficiently relevant material found', the UI should not show an empty or confusing sources expander — check for that string and skip rendering the expander entirely in that case
- Test with at least one question that should trigger retrieval and one that should not (a general knowledge question unrelated to the curriculum), and confirm the sources expander only appears when appropriate

12. Middleware: Guardrails and Context Engineering

What middleware adds on top of `create_agent`

Middleware hooks into specific points of the agent loop (before the model is called, after a tool runs, before the agent finishes) to customize behavior without rewriting the loop itself. This is the officially recommended way in LangChain 1.0 to add guardrails, summarize long conversations, redact sensitive data, or require human approval before a risky action — replacing ad hoc, hand-rolled logic scattered through application code.

Built-in middleware: PII redaction and summarization

```
from langchain.agents import create_agent
from langchain.agents.middleware import PIIMiddleware, SummarizationMiddleware

agent = create_agent(
    model="anthropic:claude-sonnet-5",
    tools=[...],
    middleware=[
        PIIMiddleware("email", strategy="redact", apply_to_input=True),
        SummarizationMiddleware(
            model="anthropic:claude-haiku-4-5",  # a cheaper model for the summarization step
            trigger={"tokens": 3000},
        ),
    ],
)
```

Human-in-the-loop for risky actions

For a tool that sends a real email, modifies a database, or takes any consequential action, `HumanInTheLoopMiddleware` pauses the agent before that specific tool executes and waits for explicit approval, edit, or rejection — critical for any agent given real-world side effects rather than read-only tools.

```
from langchain.agents.middleware import HumanInTheLoopMiddleware

agent = create_agent(
    model="anthropic:claude-sonnet-5",
    tools=[send_announcement_email, search_curriculum],
    middleware=[
        HumanInTheLoopMiddleware(
            interrupt_on={"send_announcement_email": {"allowed_decisions": ["approve", "edit",
"reject"]}}
        ),
    ],
)
```

Writing custom middleware

For guardrails specific to your domain, subclass `AgentMiddleware` and implement one of its lifecycle hooks. A simple example: rejecting questions that fall outside the bootcamp's scope before the model spends a call on them.

```
from langchain.agents.middleware import AgentMiddleware

class ScopeGuardMiddleware(AgentMiddleware):
    BLOCKED_TOPICS = ["medical advice", "legal advice", "financial investment advice"]

    def before_agent(self, state):
        last_message = state["messages"][-1].content.lower()
        if any(topic in last_message for topic in self.BLOCKED_TOPICS):
            return {"messages": [{"role": "assistant",
                                   "content": "I can only help with bootcamp-related questions."}]}
        return None  # None means: continue normally, no interruption
```

Exercise

EXERCISE — Try it yourself first

Add `PIIMiddleware` to redact email addresses from user input before it reaches the model, using the curriculum agent from Chapter 11. Test it with a message that includes a fake email address and confirm the redaction happens by inspecting the message that was actually sent to the model (not just the final answer).

Worked Solution

SOLUTION

```
from langchain.agents import create_agent
from langchain.agents.middleware import PIIMiddleware
from core.tools import search_curriculum, get_module_deadline

agent = create_agent(
    model="anthropic:claude-sonnet-5",
    tools=[search_curriculum, get_module_deadline],
    system_prompt="You are a bootcamp assistant.",
    middleware=[PIIMiddleware("email", strategy="redact", apply_to_input=True)],
)

result = agent.invoke({
    "messages": [{"role": "user", "content": "My email is student.test@example.com, when is CNN due?"}]
})

# Inspect the full message trace to see the redacted version, not just the final answer:
for msg in result["messages"]:
    print(type(msg).__name__, "-", getattr(msg, "content", msg))
```

Printing the full messages list rather than only messages[-1] is essential to verify middleware behavior — the redaction happens on the message actually sent to the model, and if you only look at the final assistant reply, you cannot confirm the model never saw the raw email address.

Mini Project: Guardrailed Curriculum Assistant

MINI PROJECT

Harden the Chapter 11 curriculum assistant with layered middleware, moving from 'it works' to 'it is safe to hand to real students.'

Requirements:

- core/agents.py: add PII Middleware for email and phone number redaction on input
- core/agents.py: add SummarizationMiddleware with a reasonable token trigger so very long conversations do not silently fail once they exceed the model's context window
- core/middleware.py: a custom ScopeGuardMiddleware (following the pattern shown above) that politely declines questions clearly unrelated to the bootcamp, with your own list of out-of-scope keywords appropriate for an education context
- pages/4_Curriculum_Assistant.py: no changes required to the page itself — this is the point of middleware, the UI layer stays untouched while the agent's safety behavior improves in core/
- Manually test all three middleware behaviors with example messages and note the results in a short markdown file (docs/middleware_test_notes.md), not part of the app itself

13. MCP: Connecting External Tool Servers

What MCP adds beyond ordinary LangChain tools

Every tool covered so far was a Python function you wrote and registered by hand. The Model Context Protocol (MCP) is an open standard for exposing tools, resources, and prompts from a separate server process, so an agent can discover and use capabilities it was never explicitly coded to know about — a database server, a filesystem server, a company's internal API — all through one consistent protocol instead of a custom integration per service.

langchain-mcp-adapters is the official bridge: it converts MCP tools into LangChain-compatible tools automatically, so once connected, MCP tools are used by `create_agent` exactly like the `@tool`-decorated functions from Chapter 6.

Connecting to one or more MCP servers

```
pip install langchain-mcp-adapters
```

MultiServerMCPClient

```
from langchain_mcp_adapters.client import MultiServerMCPClient
from langchain.agents import create_agent

client = MultiServerMCPClient({
    "curriculum_db": {
        "command": "python",
        "args": ["/path/to/curriculum_mcp_server.py"],
        "transport": "stdio",
    },
    "web_search": {
        "url": "http://localhost:8000/mcp",
        "transport": "http",
    },
})

tools = await client.get_tools() # discovers and converts every tool from both servers
agent = create_agent("anthropic:claude-sonnet-5", tools)

response = await agent.ainvoke({
    "messages": [{"role": "user", "content": "Search the curriculum database for RAG-related modules."}]
})
print(response["messages"][-1].content)
```

Combining MCP tools with your own hand-written tools

MCP tools and native LangChain tools are both just items in the tools list passed to `create_agent` — there is no special handling needed to mix them. This means you can add a growing ecosystem of MCP servers to an agent without touching your existing `core/tools.py` functions.

```
from core.tools import search_curriculum, get_module_deadline

mcp_tools = await client.get_tools()
all_tools = mcp_tools + [search_curriculum, get_module_deadline]
agent = create_agent("anthropic:claude-sonnet-5", all_tools)
```

Async and Streamlit

MCP client calls are async (`get_tools()`, `agent.ainvoke()`), while most of this guide's Streamlit examples have used the synchronous `.invoke()`. Streamlit pages can call async code by wrapping it with `asyncio.run()` at the point of use, which is the pattern the mini project below applies.

Exercise

EXERCISE — Try it yourself first

Write a minimal local MCP server (using the official mcp Python package) exposing a single tool, `get_current_time(timezone: str) -> str`. Connect to it with `MultiServerMCPClient` using `stdio` transport, load its tools, and confirm an agent built with those tools can correctly answer 'What time is it in Douala right now, roughly?' by calling the tool (note: a simple demo tool can return a fixed or approximate value — the point is proving the wiring works end to end).

Worked Solution

SOLUTION

```
# time_server.py – a minimal MCP server
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("time_server")

@mcp.tool()
def get_current_time(timezone: str) -> str:
    """Return an approximate current time description for a given timezone name."""
    return f"It is currently daytime in {timezone} (demo server – not a real clock)."
```

```
if __name__ == "__main__":
    mcp.run(transport="stdio")

# agent_test.py – connecting to it
import asyncio
from langchain_mcp_adapters.client import MultiServerMCPClient
from langchain.agents import create_agent
```

```

async def main():
    client = MultiServerMCPClient({
        "time": {"command": "python", "args": ["time_server.py"], "transport": "stdio"},
    })
    tools = await client.get_tools()
    agent = create_agent("anthropic:claude-sonnet-5", tools)
    result = await agent.ainvoke({
        "messages": [{"role": "user", "content": "What time is it in Douala right now, roughly?"}]
    })
    print(result["messages"][-1].content)

asyncio.run(main())

```

The MCP server is a completely separate process from the agent — langchain-mcp-adapters starts it (for stdio transport) and communicates over standard input/output using the MCP protocol's message format. This process boundary is the entire point: the server could be written in any language that implements MCP, not just Python, and the agent code would not change.

Mini Project: MCP-Connected Research Assistant

MINI PROJECT

Extend the SEED assistant with a real external MCP server rather than a toy demo — connecting to a public or self-hosted MCP server for web search or file access, alongside the existing curriculum and operations tools.

Requirements:

- `core/mcp_client.py`: a `get_mcp_tools()` async function wrapping `MultiServerMCPClient` configuration, kept separate from `core/tools.py` (which holds native tools) to preserve the separation between MCP-sourced and hand-written tools
- `core/agents.py`: a `build_research_agent()` that combines MCP tools with `search_curriculum` and `get_module_deadline` into one agent
- `pages/5_Research_Assistant.py`: a chat page that calls the async agent using `asyncio.run()` at the point of invocation, with a clear code comment explaining why (Streamlit's execution model is synchronous)
- Handle MCP connection failures gracefully: if the MCP server is unreachable, catch the exception and show an `st.warning()` explaining that external tools are unavailable, while still allowing the native tools to work
- Document, in a short comment block at the top of `core/mcp_client.py`, exactly which MCP server(s) this connects to and what each exposes, since MCP's dynamic tool discovery means the available tools are not otherwise visible just by reading `core/tools.py`

14. Multi-Agent Systems and Streaming in Streamlit

When one agent is not the right shape

A single agent with many tools and a long system prompt works up to a point, but tends to get confused as tool count and instruction complexity grow. A multi-agent architecture splits responsibility: a supervisor agent routes requests to specialist sub-agents, each with a focused tool set and system prompt, and the supervisor is the only one that talks directly to the user.

A supervisor pattern using `create_agent` as a tool

The simplest way to build a supervisor in LangChain: wrap each specialist agent's `.invoke()` call in a `@tool` function, and give the supervisor those wrapped agents as its own tools. The supervisor's model then decides which specialist to delegate to, exactly as it would decide which ordinary tool to call.

```
from langchain_core.tools import tool
from langchain.agents import create_agent
from core.tools import search_curriculum, get_module_deadline, count_students_in_module

curriculum_agent = create_agent(
    model="anthropic:claude-haiku-4-5",
    tools=[search_curriculum],
    system_prompt="You answer questions about curriculum content only, citing sources.",
)
ops_agent = create_agent(
    model="anthropic:claude-haiku-4-5",
    tools=[get_module_deadline, count_students_in_module],
    system_prompt="You answer operational questions about deadlines and enrollment only.",
)

@tool
def ask_curriculum_specialist(question: str) -> str:
    """Delegate a question about curriculum content to the curriculum specialist."""
    result = curriculum_agent.invoke({"messages": [{"role": "user", "content": question}]})
    return result["messages"][-1].content

@tool
def ask_ops_specialist(question: str) -> str:
    """Delegate a question about deadlines or enrollment to the operations specialist."""
    result = ops_agent.invoke({"messages": [{"role": "user", "content": question}]})
    return result["messages"][-1].content

supervisor = create_agent(
    model="anthropic:claude-sonnet-5",
    tools=[ask_curriculum_specialist, ask_ops_specialist],
    system_prompt="Route each question to the correct specialist tool, or answer directly if it's simple chit-chat.",
)
```

Streaming an agent's full trace into Streamlit

For a multi-agent system especially, showing only the final answer hides useful structure. `astream_events` (or the synchronous `stream_events`, depending on version) exposes granular events — model tokens, tool starts, tool ends — which can drive a live 'agent is working' UI in Streamlit using `st.status`.

```
import streamlit as st

with st.status("Working on it...", expanded=True) as status:
    for step in supervisor.stream({"messages": [{"role": "user", "content": user_question}]}):
        for node_name, node_output in step.items():
            st.write(f"Step: {node_name}")
            status.update(label="Done", state="complete")

final = supervisor.invoke({"messages": [{"role": "user", "content": user_question}]}))
st.markdown(final["messages"][-1].content)
```

Exercise

EXERCISE — Try it yourself first

Using the supervisor pattern above, ask the supervisor a question that clearly needs the operations specialist ('How many students are in ANN?'), one that clearly needs the curriculum specialist ('Explain transfer learning'), and one ambiguous question that could go either way ('Tell me about the CNN module'). Print which specialist tool was actually called for each, by inspecting the supervisor's returned message trace for `tool_calls`.

Worked Solution

SOLUTION

```
questions = [
    "How many students are in ANN?",
    "Explain transfer learning",
    "Tell me about the CNN module",
]

for q in questions:
    result = supervisor.invoke({"messages": [{"role": "user", "content": q}]}))
    tool_calls_made = [
        msg.tool_calls[0]["name"]
        for msg in result["messages"]
        if hasattr(msg, "tool_calls") and msg.tool_calls
    ]
    print(f"Question: {q}")
    print(f"  Specialist(s) called: {tool_calls_made or 'none (answered directly)'}")
    print(f"  Final answer: {result['messages'][-1].content[:100]}...")
```

The ambiguous question is the interesting case to actually run: the supervisor's choice depends entirely on its system prompt and its own judgment about what 'tell me about the CNN module' means, which may vary between runs at nonzero temperature — this variability is expected in agentic routing and is exactly why testing with real example questions matters more than assuming behavior from the code alone.

Mini Project: Multi-Agent SEED Assistant with Live Status

MINI PROJECT

The most complete assembly in this guide: a supervisor-routed, multi-agent Streamlit page with live progress, combining nearly every prior chapter's building blocks.

Requirements:

- `core/agents.py`: `curriculum_agent` and `ops_agent` as focused specialists, plus a `build_supervisor_agent()` combining them via the delegation-tool pattern, with a checkpoint for conversation memory
- `pages/6_Multi_Agent_Assistant.py`: a chat page using `st.status` to show which specialist is being consulted in real time, using the streaming pattern shown above
- After the final answer, show a small caption noting which specialist(s) handled the question, extracted the same way as the exercise
- Reuse the `PIIMiddleware` and `ScopeGuardMiddleware` from Chapter 12 on the supervisor agent specifically (specialists can stay unguarded since they never see raw user input directly, only the supervisor's delegated question)
- Write a short comparison note (as an `st.expander` on the page itself, labeled 'Why multiple agents?') explaining in 2-3 sentences when this pattern is worth the added complexity versus a single agent with many tools

15. Production Architecture and Deployment

Auditing the separation of concerns one last time

By this chapter, the `genai_app` project should have a `core/` package that is entirely UI-independent, and `pages/` files that mostly wire `core/` functions to Streamlit widgets. Before deployment, verify this boundary actually held: nothing in `core/` should import streamlit, and every `core/` function should be callable — and testable — from a plain Python script with no browser involved.

Testing agent logic without live API calls

Full end-to-end tests that call a real model are slow and cost money to run on every commit. Structure most tests around the parts that do not require a live call: that tools return correct values given fixed inputs, that prompt templates fill in variables correctly, and that a chain or agent object is constructed without error. Reserve a small number of real, live-call tests for critical paths, run less frequently (e.g. before a release, not on every save).

```
# tests/test_tools.py
from core.tools import get_module_deadline, count_students_in_module

def test_get_module_deadline_known_module():
    result = get_module_deadline.invoke({"module_name": "CNN"})
    assert result == "2026-09-15"

def test_get_module_deadline_unknown_module():
    result = get_module_deadline.invoke({"module_name": "Not A Real Module"})
    assert "unknown" in result.lower() or "not found" in result.lower()
```

Secrets and configuration for deployment

Exactly as in the Streamlit-only guide: secrets live in `.streamlit/secrets.toml` locally and in the Streamlit Community Cloud dashboard when deployed, never hardcoded. LangChain's model classes read provider API keys from environment variables by default (`ANTHROPIC_API_KEY`, `OPENAI_API_KEY`), so setting them via `st.secrets` and `os.environ` at app startup covers both LangChain's needs and any other library's.

```
# app.py, near the top, before any core/ import that builds a model
import os
import streamlit as st

os.environ.setdefault("ANTHROPIC_API_KEY", st.secrets.get("ANTHROPIC_API_KEY", ""))
```

Production checklist specific to LangChain apps

- Every `agent.invoke()` / `chain.invoke()` call in a page is wrapped in `try/except` with a specific, user-facing error message

- Checkpointer choice matches the deployment: InMemorySaver for local dev only; a persistent backend (e.g. a Postgres-backed checkpointer) for anything users expect to survive a restart
- Token and cost awareness: SummarizationMiddleware or an explicit history-trimming strategy is in place for any agent expected to hold long conversations
- Tool docstrings have been reviewed for clarity, since a vague docstring silently degrades an agent's tool-selection accuracy in ways that are easy to miss in casual testing
- MCP server connections (if used) fail gracefully with the app still functional using native tools only
- Vector store ingestion (Chapter 10) runs as a separate, explicit script — never automatically inside the Streamlit app on every restart
- requirements.txt pins exact versions for langchain, langchain-core, langgraph, and every provider package, since this ecosystem changes quickly and an unpinned deploy can break without any code change on your part

Exercise

EXERCISE — Try it yourself first

Write two pytest tests for the `search_curriculum` tool from Chapter 11: one confirming it returns the 'No sufficiently relevant material found' message for a nonsense query unrelated to any curriculum content, and one confirming it returns text containing '[Source:]' for a query you know matches your indexed documents. Run both without needing a live model call (the tool itself only calls the vector store, not the LLM).

Worked Solution

SOLUTION

```
# tests/test_retrieval_tool.py
from core.tools import search_curriculum

def test_search_curriculum_no_match():
    result = search_curriculum.invoke({"query": "xyzzy quantum toaster nonsense query"})
    assert "no sufficiently relevant material" in result.lower()

def test_search_curriculum_finds_source():
    result = search_curriculum.invoke({"query": "transfer learning"})
    assert "[Source:" in result
```

These tests only exercise the vector store's similarity search, not the LLM itself, which is why they run quickly, deterministically, and with no API key required — this is exactly the kind of test to run on every commit, reserving live-model tests for a smaller, separate suite.

Capstone Mini Project: Production-Ready SEED GenAI Assistant

MINI PROJECT

The final assembly of the entire guide: take the full `genai_app` project — chat, feedback triage, operations agent, curriculum agentic RAG, middleware guardrails, MCP tools, and the multi-agent supervisor — and prepare it as one coherent, deployable product.

Requirements:

- Run a full audit: `grep` every file in `core/` for `'import streamlit'` and confirm there are zero matches; fix any violations found
- Write at least 8 `pytest` tests across `tests/test_tools.py`, `tests/test_chains.py`, and `tests/test_retrieval_tool.py`, none of which require a live API key to run
- Add a shared `core/errors.py` with a single `safe_invoke(callable, *args, fallback_message, **kwargs)` helper used consistently across every page instead of repeating `try/except` blocks
- Finalize `requirements.txt` with pinned versions for every LangChain-ecosystem package actually used across the whole project
- Write a `README.md` (plain markdown, not part of the running app) documenting: the folder structure and what belongs where, how to set up secrets locally, how to run `scripts/build_index.py` before first use, and how to deploy to Streamlit Community Cloud
- Add a landing `app.py` that lists all 6 pages with a one-line description of each, so a new user (or a bootcamp cohort seeing this for the first time) understands the full scope of the tool before diving into any one page

Where to Go From Here

By completing every mini project in this guide, you will have built `genai_app` end to end: a persona-driven chat page, structured feedback triage, a tool-calling operations agent, an agentic RAG curriculum assistant with cited sources, layered middleware guardrails, an MCP-connected research assistant, and a multi-agent supervisor system — all sitting on a `core/` package with zero Streamlit imports, backed by a real test suite. That is close to the actual shape of a production GenAI application, not a simplified version of one.

Suggested next steps

- Swap `InMemorySaver` for a persistent checkpointer (e.g. a Postgres-backed one) so conversations survive an app restart in a real deployment
- Swap the local HuggingFace embeddings for Voyage AI (Anthropic's recommended embeddings partner) for production-quality retrieval
- Add LangSmith tracing to see the full execution trace of every agent run in a dashboard, which becomes essential once middleware and multi-agent routing make behavior harder to reason about from logs alone
- Explore LangGraph directly (the lower-level framework `create_agent` is built on) once you need agent topologies more custom than a simple supervisor — branching, cycles, or parallel sub-agent execution
- Add a second MCP server from the growing public ecosystem (e.g. a filesystem or database server) and confirm your agent can use it with zero changes to `core/agents.py`, only to the MCP client configuration
- Deploy `genai_app` to Streamlit Community Cloud following the deployment chapter of the companion Streamlit guide, adapted for LangChain's secrets needs (Chapter 15 of this guide)

Real proficiency comes from the mini projects and from reading current LangChain documentation directly once you hit something this guide does not cover — the framework moves quickly, and the separation-of-concerns habits built here are what make it safe to upgrade one piece (a model, a vector store, a checkpointer) without rewriting the whole project.